

Week 11: Seeing with Networks: Convolutional Neural Networks and Computer Vision

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Web version with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-11/lecture.html>

Colab notebook: https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-11/NB11_cnn_vision.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: September 2026.

Overview

Cameras are cheap sensors, and much engineering knowledge is visual: cracks in concrete, defects on steel strip, grains on a conveyor, fibres in a fabric, cells under a microscope. Convolutional neural networks learn visual features directly from pixels by sharing small filters across the whole image [1, 2]. Since AlexNet won the ImageNet challenge, deep convolutional networks and their successors have transformed computer vision [3, 4, 5]. This week explains convolution, pooling and modern architectures, trains a crack detector on generated concrete images, compares networks on a clothing benchmark, reuses a pretrained network through transfer learning [6], and asks the network where it looks with Grad-CAM [7].

Estimated study time: 10 to 12 hours.

Learning outcomes

By the end of the week, students are expected to compute a convolution and a pooling step by hand, to calculate output sizes and parameter counts of convolutional layers, to build and train a small convolutional network in PyTorch, to explain why weight sharing makes convolutional networks efficient, to apply transfer learning and data augmentation to small engineering datasets, and to inspect a trained network with Grad-CAM and judge whether it relies on the right evidence.

Python in this week

The lecture ends with Python step 11: Images, batches and network classes. It covers images as arrays, cropping and flipping, stacking images into batches, the channel order of PyTorch, iterating with a DataLoader, and networks written as classes, applied to crack images [8]. The hands-on part of the notebook opens with the same step as runnable cells and short exercises with immediate feedback, and its later sections mark further constructs as Python moves.

Images as engineering data

A digital image is an array of numbers. A greyscale image of 64 by 64 pixels holds 4096 intensities, and a colour image stores three such arrays, one per channel. Engineering images come from inspection cameras on production lines, drones over bridges and power lines, microscopes, satellites, thermal cameras and X-ray systems. Treating the pixels as 4096 unrelated inputs to a multilayer network ignores two facts: Neighbouring pixels belong together, and the same pattern, such as an edge or a crack, can appear anywhere in the image. Convolutional networks build both facts into their structure.

Convolution, feature maps and pooling

A convolutional layer slides a small filter, for example 3 by 3 weights, over the image and computes at every position the weighted sum of the pixels under it. The result is a feature map that is large where the image locally resembles the filter. A vertical-edge filter responds to vertical edges wherever they are, because the same weights are used at every position. This weight sharing drastically reduces the number of parameters compared with a fully connected layer and makes the detector equivariant to shifts [1, 2]. A layer learns many filters at once, each producing its own feature map, or channel.

Three settings shape a convolution. The kernel size sets the local window, the stride sets the step between positions, and padding adds a border so that the output can keep the input size. For an input of width W , kernel K , padding P and stride S , the output width is $(W - K + 2P) / S + 1$. Pooling layers then summarise small neighbourhoods, usually by their maximum, which halves the resolution and makes the representation more tolerant to small shifts. Stacking convolution, activation and pooling lets later layers see larger parts of the image: Early layers detect edges and textures, later layers combine them into parts and objects [9].

Animation: A filter slides over an image. The highlighted 3 by 3 window moves across a small concrete image with a crack. Each output value is the sum of the pixels in the window multiplied by the filter weights. Choose different filters and compare the feature maps. Run it in the [web version of the lecture](#).

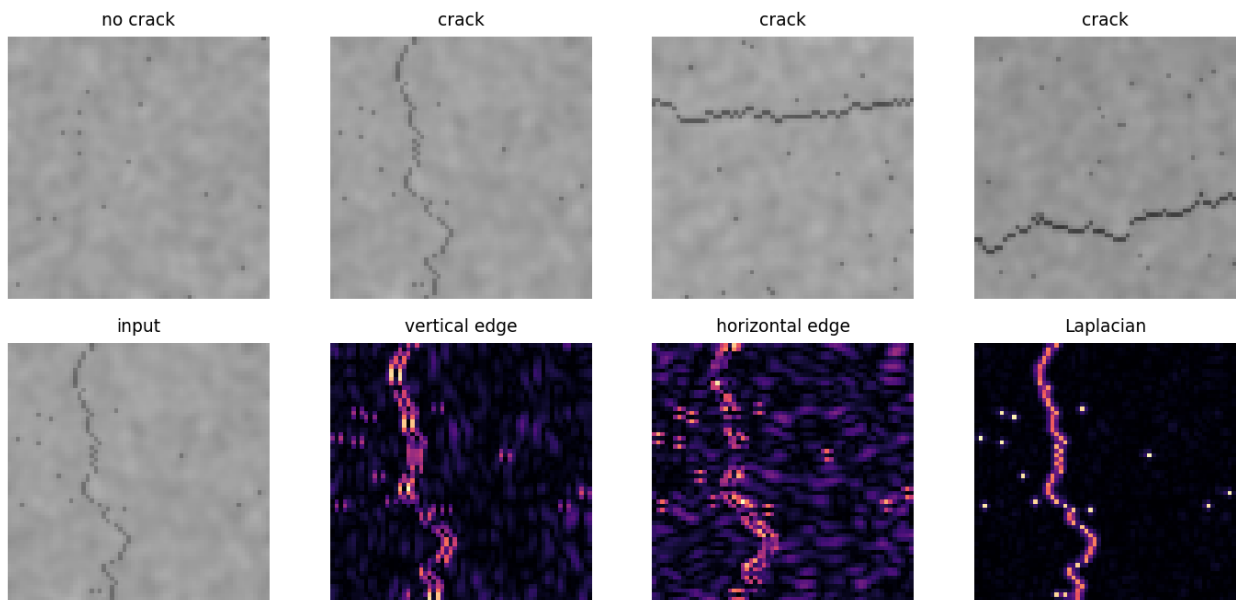


Figure 11.1. Generated concrete surfaces with and without cracks (top) and their responses to a vertical-edge filter, a horizontal-edge filter and a Laplacian filter (bottom).

Check your understanding. An input of 64 by 64 pixels passes a 5 by 5 convolution without padding and with stride 1, followed by 2 by 2 max pooling. What is the output size?

- A. 64 by 64
- B. 60 by 60

- C. 30 by 30
- D. 32 by 32

Answer: C. The convolution gives $(64 - 5 + 0) / 1 + 1 = 60$, and pooling halves it to 30.

Deep architectures

LeNet showed in the 1990s that convolutional networks trained by backpropagation read handwritten digits reliably [2]. AlexNet, a deeper network trained on graphics processors with ReLU activations and dropout, won the 2012 ImageNet challenge by a wide margin [3, 4]. VGG networks used only small 3 by 3 filters stacked deeply [10]. Residual networks added shortcut connections that let a layer learn a correction to its input, which made networks with more than a hundred layers trainable [5]. Vision transformers split an image into patches and process them with the attention mechanism of Week 13; with enough data they match or exceed convolutional networks [11].

Training with little data

Engineering datasets are small compared with ImageNet: a few hundred labelled crack photographs, a few thousand defect images. Two techniques make deep networks usable nonetheless. Data augmentation creates plausible variants of the training images by flipping, rotating, cropping or changing brightness, which teaches invariances that the task requires. Transfer learning starts from a network pretrained on a large dataset and retrain only its last layers or fine-tunes all of them with a small learning rate; Yosinski and colleagues showed that early layers learn general features that transfer well between tasks [6]. Cha and colleagues trained a convolutional network on patches of concrete photographs to detect cracks under varied lighting [12], and similar studies inspect steel strip surfaces [13], crops and weeds [14] and traffic signs [15].

Check your understanding. A team has 400 labelled images of weld defects. Which approach is most promising?

- A. Train a very deep network from random initial weights without augmentation
- B. Fine-tune a network pretrained on a large image dataset, with data augmentation
- C. Use a multilayer perceptron on raw pixels
- D. Remove the test set to have more training data

Answer: B. Pretrained features and augmentation compensate for the small dataset [6].

Beyond classification

Classification assigns one label to an image. Object detection finds and labels every object with a bounding box; YOLO treats detection as a single regression problem and runs in real time [16]. Semantic segmentation labels every pixel; U-Net combines a contracting path with an expanding path and skip connections and became a standard for segmentation with few training images [17]. For a bridge inspector, a classifier says “crack present”, a detector draws a box around each crack, and a segmentation network outlines the crack so that its length and width can be measured.

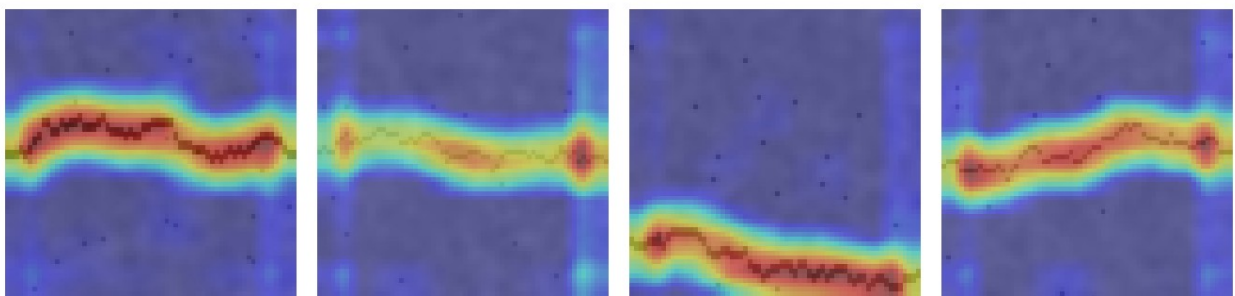


Figure 11.2. Grad-CAM heatmaps of the crack classifier trained in the notebook. The network attends to the crack pixels rather than to the background texture [7].

Seeing what the network sees

High accuracy does not prove that a network uses the right evidence. A defect classifier may learn the lighting of the station where defective parts were photographed, or the ruler that appears only in images of damaged specimens. Grad-CAM weighs the feature maps of the last convolutional layer by the gradient of the class score and produces a coarse heatmap of the regions that drove the decision [7]. Checking such maps on a sample of test images, and testing the network on images from new cameras and sites, belongs to every engineering deployment. Vision models are also vulnerable to small, deliberate perturbations of the input that change their decisions, which matters wherever an attacker could manipulate the images [18].

Check your understanding. A crack classifier reaches 99 percent test accuracy, but Grad-CAM shows that it attends to a timestamp printed in the corner of the images. What should be concluded?

- A. The classifier is excellent and can be deployed
- B. The classifier may rely on a shortcut that happens to correlate with the label, so the data and the evaluation must be revised
- C. Grad-CAM is broken
- D. The images should be larger

Answer: B. The explanation reveals a spurious cue; accuracy on data that share the cue does not guarantee performance elsewhere.

Python step 11: Images, batches and network classes

Images are arrays

A greyscale image is a two-dimensional array of intensities, usually scaled to the range 0 to 1, with the shape (height, width). Colour images add a third axis for the red, green and blue channels. Everything from Weeks 4 and 9 applies: Slicing crops an image, reversing an axis mirrors it, and masks find the pixels that meet a condition. The practice image below is light grey with a dark vertical crack in column 4.

```
import numpy as np

img = np.full((8, 10), 0.7)           # light grey image, 8 rows and 10
columns
img[:, 4] = 0.1                       # a dark crack in column 4
print(img.shape, img.min(), img.max())
crop = img[2:6, 3:6]                  # rows 2 to 5, columns 3 to 5
print(crop)
print("dark pixels:", int((img < 0.3).sum()))
flipped = img[:, ::-1]                # mirror from left to right
print("crack column after flipping:",
int(np.argmax(flipped.mean(axis=0))))
```

Output

```
(8, 10) 0.1 0.7
[[0.7 0.1 0.7]
 [0.7 0.1 0.7]
 [0.7 0.1 0.7]
 [0.7 0.1 0.7]]
dark pixels: 8
crack column after flipping: 5
```

The slice `::-1` steps backwards through an axis. Flips and small rotations are the data augmentation of this week: They create new training images that are still valid examples of a crack.

From images to batches

Networks process several images at once. `np.stack` joins images of equal shape along a new first axis, which gives the shape (number of images, height, width). PyTorch convolution layers expect a channel axis in second place, (batch, channels, height, width), so greyscale images receive an axis of length one with `[:, None]`. Libraries that read colour image files usually return (height, width, channels), and `permute` reorders the axes of a tensor.

```
import torch

batch = np.stack([img, flipped, img * 0.9])           # three images
print(batch.shape)
xb = torch.tensor(batch[:, None], dtype=torch.float32)
print(xb.shape)
rgb = torch.zeros(64, 64, 3)                         # colour image as
stored in files
print(rgb.permute(2, 0, 1).shape)                   # channels first,
as PyTorch expects
```

Output

```
(3, 8, 10)
torch.Size([3, 1, 8, 10])
torch.Size([3, 64, 64])
```

Iterating in batches

A `DataLoader` serves a dataset in batches of a chosen size, optionally in shuffled order, and a `for` loop receives one batch of images and labels per pass. `enumerate` numbers the batches. The last batch is smaller when the batch size does not divide the number of examples.

```
from torch.utils.data import TensorDataset, DataLoader

images = torch.rand(10, 1, 8, 10)
labels = torch.tensor([0, 1] * 5)
loader = DataLoader(TensorDataset(images, labels), batch_size=4,
shuffle=False)
for i, (xb_i, yb_i) in enumerate(loader):
    print(i, tuple(xb_i.shape), yb_i.tolist())
```

Output

```
0 (4, 1, 8, 10) [0, 1, 0, 1]
1 (4, 1, 8, 10) [0, 1, 0, 1]
2 (2, 1, 8, 10) [0, 1]
```

The expression `[0, 1] * 5` repeats a list five times, the list behaviour that NumPy arrays replace with element-wise arithmetic.

A network is a class

Week 6 introduced classes. A PyTorch network is a class that inherits from `nn.Module`: It gains all the machinery of a network and adds its own layers in `__init__` and its computation in `forward`. The call `super().__init__()` first runs the initialisation of the parent class. Calling the object, as in `net(xb)`, runs `forward`.

```
from torch import nn
```

```

class TinyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 4, kernel_size=3, padding=1)
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(4, 2)

    def forward(self, x):
        x = torch.relu(self.conv(x))
        return self.fc(self.pool(x).flatten(1))

```

```

net = TinyCNN()
print(net(xb).shape)
print(sum(p.numel() for p in net.parameters()), "parameters")

```

Output

```

torch.Size([3, 2])
50 parameters

```

The convolution has 4 filters of 3 by 3 weights plus 4 biases, 40 parameters, and the linear layer has 4 times 2 weights plus 2 biases, 10 parameters.

Check your understanding. Which shape does PyTorch expect for a batch of 16 colour images of 32 by 32 pixels?

- A. (16, 32, 32, 3)
- B. (16, 3, 32, 32)
- C. (3, 16, 32, 32)
- D. (32, 32, 3, 16)

Answer: B. Convolution layers expect batch, channels, height and width, in this order.

Discipline challenges

Images are part of almost every discipline. Choose a row and adapt the notebook's crack classifier or its transfer-learning section to the images named there.

Department	Challenge
Civil Engineering	Crack detection on concrete or pavement photographs, following Cha and colleagues [12].
Mechanical Engineering and Textile Engineering	Surface defect classes on hot-rolled steel strip or fabric [13].
Food Engineering and Biology	Grain, fruit or leaf disease images with transfer learning [14].
Automotive Engineering and Computer Engineering	Traffic sign recognition with the German benchmark [15].
Geological Engineering and Earth Sciences Engineering	Rock thin-section or outcrop images classified by lithology.
Environmental Engineering	Satellite or drone image tiles classified by land cover or pollution events.
Mining Engineering	Fragmentation images after blasting, classified by size class.
Electrical and Electronics Engineering	Printed circuit board defect images or thermal images of electrical panels.

Department	Challenge
Physics and Chemistry	Microscopy images of materials or crystals classified by phase or morphology.
Mathematics and Statistics	Show that convolution is linear and shift-equivariant and that max pooling is neither linear nor equivariant.

Weekly task

Train and evaluate a convolutional network for an image task related to your department. Use the generated crack images, the clothing or shape benchmark of the notebook, or a public engineering image dataset whose licence allows teaching use. Report the architecture, parameter count, augmentation, learning curves and test metrics, compare training from scratch with transfer learning, and show Grad-CAM maps for correct and wrong predictions. Discuss in about 500 words what would be needed for use on real site images, citing at least three works from this week's references [5, 6, 7].

The weekly task is optional and supports self-learning and a personal portfolio. During an active semester, it can be sent together with the exported learning log to utkukose@sdu.edu.tr or utkukose@gmail.com. Optional work carries no separate weight, but the instructor may take it into account as a discretionary adjustment of the midterm component of the grade.

Research and report assignment (optional)

Computer vision for inspection. Review deep learning for visual inspection in one domain, such as concrete and pavement cracks, steel surface defects, fabric defects, agricultural products or electronics. Compare datasets, architectures, the use of transfer learning and augmentation, and how the studies tested generalisation to new sites or cameras [12, 13, 14].

References

- [1] LeCun, Y., Boser, B., Denker, J. S., et al. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4), 541-551. <https://doi.org/10.1162/neco.1989.1.4.541>
- [2] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324. <https://doi.org/10.1109/5.726791>
- [3] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). ImageNet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6), 84-90. <https://doi.org/10.1145/3065386>
- [4] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248-255). <https://doi.org/10.1109/CVPR.2009.5206848>
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770-778). <https://doi.org/10.1109/CVPR.2016.90>
- [6] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks?. In *Advances in Neural Information Processing Systems 27* (pp. 3320-3328). <https://arxiv.org/abs/1411.1792>
- [7] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *2017 IEEE International Conference on Computer Vision (ICCV)* (pp. 618-626). <https://doi.org/10.1109/ICCV.2017.74>
- [8] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024-8035). <https://arxiv.org/abs/1912.01703>

- [9] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
<https://doi.org/10.1038/nature14539>
- [10] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations (ICLR 2015)*.
<https://arxiv.org/abs/1409.1556>
- [11] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations (ICLR 2021)*.
<https://arxiv.org/abs/2010.11929>
- [12] Cha, Y.-J., Choi, W., & Büyüköztürk, O. (2017). Deep learning-based crack damage detection using convolutional neural networks. *Computer-Aided Civil and Infrastructure Engineering*, 32(5), 361-378.
<https://doi.org/10.1111/mice.12263>
- [13] Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, 285, 858-864.
<https://doi.org/10.1016/j.apsusc.2013.09.002>
- [14] Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147, 70-90. <https://doi.org/10.1016/j.compag.2018.02.016>
- [15] Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32, 323-332.
<https://doi.org/10.1016/j.neunet.2012.02.016>
- [16] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779-788). <https://doi.org/10.1109/CVPR.2016.91>
- [17] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*, LNCS 9351 (pp. 234-241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28
- [18] Kose, U. (2019). Techniques for adversarial examples threatening the safety of artificial intelligence based systems. In *I. International Science and Innovation Congress (INSI Congress 2019)*. Pamukkale, Denizli, Türkiye. <https://arxiv.org/abs/1910.06907>